Day 83: The Week 12 Build — a Tiny Transformer From the Exercise Parts, Trained

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 12, Day 7 | Complete study guide

Watch: Coding a Transformer from scratch, training and inference (Umar Jamil) | Let's build GPT: from scratch, in code, spelled out (Andrej Karpathy)

Script: `day83_tiny_transformer.py` · every number in this guide is copied by program from `day83_run_output.txt`, the real output of that script.

Six exercise scripts, one per day, each built one part of the Transformer. Today they are put together without changes and trained exactly as Day 71 trained its attention LSTM, on the same COPY and REVERSE digits, so the two can be compared. The Transformer learns COPY much faster: every length perfect after 1500 steps for two of three seeds, where the LSTM needed 7,500. On REVERSE it is slower and ends a little short (0.986 at 20 digits, where Day 71 reached 1.000). Past 20 digits both fail, and the Transformer's REVERSE fails completely.

0. Where today fits

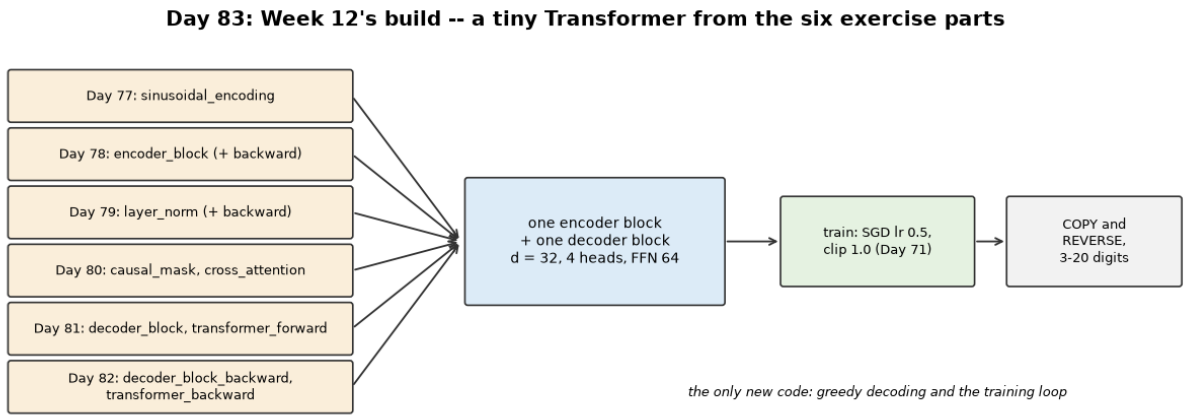
DEFINITION — start here, no prior knowledge assumed

An **integration day** joins parts that were built and checked one at a time. A part that passed its own test can still fail when joined to the others (a wrong shape, a different convention), so the joined model is checked again as a whole, then trained. **Training end to end** means every weight, from the embedding table to the output layer, is updated from the one loss at the end.

REAL-WORLD SCENARIO

A car built from parts that each passed the factory's test still goes for a test drive: the engine, gearbox and brakes have to work together, on a real road.

SYSTEM DIAGRAM



Week 12's build: six exercise parts, one trained model.

The parts

Exercise	Part	Where it goes
Day 77	sinusoidal_encoding	positions, added to the embeddings
Day 78	encoder_block (+ backward)	the encoder; its attention and heads
Day 79	layer_norm (+ backward)	every add & norm
Day 80	causal_mask, cross_attention	the decoder's two attentions
Day 81	decoder_block, transformer_forward	the decoder and the whole forward pass
Day 82	decoder_block_backward, transformer_backward	the whole backward pass

The only new code is greedy decoding and the training loop.

How to read each section

Step	What it gives you
DEFINITION	What the idea is, in plain words. Start here.
REAL-WORLD SCENARIO	An everyday situation where the same idea shows up.
MATHEMATICS	The formula, when the section introduces one.
SPECIFICATION	The exact setup of the experiment.
Why this test	The question the experiment answers.
IMPLEMENTATION	The real code from the script.
ACTUAL OUTPUT	What the script really printed, copied unchanged.
What it means	How to read the numbers, and what they do and do not show.
Why choose this? / Why NOT this?	When the idea is the right tool, and what it costs.

1. The build fits

DEFINITION — start here, no prior knowledge assumed

The assembled model, checked entry by entry against numerical gradients at tiny sizes, with padded source and target positions. Then its size at the lesson's settings, against Day 71's model.

REAL-WORLD SCENARIO

Before the test drive, every connection in the wiring loom is checked with a meter.

SPECIFICATION

Check	d = 8, 2 heads, FFN 12; 3 examples, source padded in one, target in another; every weight moved a little off its start; every entry nudged by ±1e-5; Day 62's pass rule.
Lesson's model	d = 32 (Day 71's hidden size), 4 heads, FFN 64, one encoder block and one decoder block.

Why this test

Each exercise checked its own part. This checks that the six of them, copied side by side, still give the right gradient together.

ACTUAL OUTPUT

```
the lesson's model: d = 32, FFN 64, 4 heads, one encoder block and one decoder block: 21642
    weights (Day 71's attention LSTM: 17866)
gradient check: 1442 entries, largest |numeric - analytic| = 6.60e-11, every entry passes: True
Actual output from this lesson's run.
```

What it means

All 1442 entries pass (largest difference 6.60e-11). The Transformer has 21642 weights, about a fifth more than Day 71's attention LSTM (17866): a fair comparison by size.

WHY CHOOSE THIS?

Proves the copied parts fit before hours of training.

WHY NOT THIS? (TRADEOFFS)

Tiny sizes; the real model is checked only through them.

2. COPY, trained exactly as Day 71

DEFINITION — start here, no prior knowledge assumed

The model reads 3 to 20 digits and must write them back, then EOS. Exact match: the whole output is right. Training is Day 71's: plain SGD, learning rate 0.5, gradient clipped at norm 1.0, batches of 32 sequences of a random length, 20,000 steps, three seeds, the same data stream and test sets.

REAL-WORLD SCENARIO

Two students take the same course, with the same homework in the same order, and sit the same exam.

MATHEMATICS

$$w \leftarrow w - lr \cdot \min(1, 1 / |G|) \cdot G \quad (|G| = \text{the norm of every gradient together})$$

$$loss = \text{mean over target positions of } -\log p(\text{correct token})$$

SPECIFICATION

Model	21,642 weights; output layer starts 10 times smaller (Day 81).
Measured	Exact match at 4, 8, 12, 16 and 20 digits, 300 fixed sequences each, greedy decoding, at 10 checks.

Why this test

Day 71's attention LSTM solved this task. Keeping everything else the same means any difference comes from the model.

IMPLEMENTATION

```
def train(task, seed, lr=LR, steps=STEPS):
    P = init_params(np.random.RandomState(seed), V, D, F)
    rng = np.random.RandomState(seed + 100)
    curve = []
    for step in range(1, steps + 1):
        n = rng.randint(3, 21)
        _, Xd, Y, src = make_batch(rng, 32, n, task)
        tgt_in = Xd.argmax(axis=-1)
        sm, tm = np.ones(src.shape, bool), np.ones(tgt_in.shape, bool)
        logp, cache = transformer_forward(src, tgt_in, P, HEADS, sm, tm)
        G = dict(leaves(transformer_backward(logp, Y, tm, P, HEADS, cache)))
        norm = np.sqrt(sum(np.sum(g ** 2) for g in G.values()))
        for k, A in leaves(P):
            A -= lr * min(1.0, 1.0 / norm) * G[k]
        if step in CHECKS:
            curve.append([evaluate(P, n_, task)[0] for n_ in EVAL_LENGTHS])
    return P, np.array(curve)
```

ACTUAL OUTPUT

```
  steps     n=4     n=8    n=12    n=16    n=20
    500   0.631   0.649   0.342   0.300   0.196
   1000   0.953   1.000   0.996   0.996   0.956
   1500   1.000   1.000   1.000   1.000   1.000
   2000   1.000   1.000   1.000   1.000   1.000
   2500   1.000   1.000   1.000   1.000   1.000
   5000   1.000   1.000   1.000   1.000   1.000
   7500   1.000   1.000   1.000   1.000   1.000
  10000   1.000   1.000   1.000   1.000   1.000
  15000   1.000   1.000   1.000   1.000   1.000
  20000   1.000   1.000   1.000   1.000   1.000
Day 71's attention LSTM, the same setup (from its run output): at 20 digits 0.000 after 2500
    steps, 0.992 after 5000, 1.000 from 7500
Actual output from this lesson's run.
```

ACTUAL OUTPUT

```
per seed, final exact match at 20 digits, and the first check at which every length is 1.000:
  seed 0: 1.000   all perfect from: 1500 steps
  seed 1: 1.000   all perfect from: 1000 steps
  seed 2: 1.000   all perfect from: 1500 steps
Actual output from this lesson's run.
```

What it means

After 1000 steps the Transformer is already at 0.956 on 20 digits, and every length is perfect from 1,500 steps (seeds 0 and 2) or 1,000 (seed 1). Day 71's LSTM was at 0.000 on 20 digits after 2,500 steps and 0.992 after 5,000. Two reasons are likely: attention reads any source position directly from step one, with no recurrence to learn through, and the positional encoding gives the 'same position' rule COPY needs. Neither was isolated here.

WHY CHOOSE THIS?

Much faster learning on the task attention suits best.

WHY NOT THIS? (TRADEOFFS)

One easy task; it says little about harder ones (next section).

3. REVERSE: slower, and not quite perfect

DEFINITION — start here, no prior knowledge assumed

The same training, but the output is the input backwards. Output digit j must be read from source position n – 1 – j, which depends on the length n.

REAL-WORLD SCENARIO

Reading a phone number back to the caller backwards: you need to know where it ends before you can start.

Why this test

COPY can be solved with a fixed rule, 'output position j reads source position j'. REVERSE cannot: the right source position moves with the length.

ACTUAL OUTPUT

```
  steps     n=4     n=8    n=12    n=16    n=20
    500   0.112   0.002   0.000   0.000   0.000
   1000   0.340   0.059   0.001   0.000   0.000
   1500   0.616   0.244   0.126   0.026   0.000
   2000   0.620   0.282   0.190   0.079   0.023
   2500   0.782   0.601   0.481   0.328   0.080
   5000   0.968   0.968   0.954   0.900   0.773
   7500   0.976   0.979   0.969   0.843   0.844
  10000   0.999   0.976   0.978   0.966   0.952
  15000   1.000   0.996   0.998   0.990   0.979
  20000   1.000   0.998   0.999   0.992   0.986
Actual output from this lesson's run.
```

ACTUAL OUTPUT

```
per seed, final exact match at 20 digits, and the first check at which every length is 1.000:
  seed 0: 0.983   all perfect from: never
  seed 1: 1.000   all perfect from: 15000 steps
  seed 2: 0.973   all perfect from: never
Actual output from this lesson's run.
```

DAY 71'S OUTPUT, FOR COMPARISON (day71_run_output.txt, section 4)

```
model                                    n=4     n=8    n=12    n=16    n=20  weights
copy, hidden 32 (Section 3)            1.000   1.000   1.000   1.000   1.000    17866
copy, hidden 16                        1.000   1.000   1.000   1.000   1.000     5898
reverse, hidden 32                     1.000   1.000   1.000   1.000   1.000    17866
```

What it means

The Transformer learns REVERSE far more slowly than COPY (0.080 at 20 digits after 2,500 steps) and ends at 0.986. Only seed 1 gets every length perfect, at 15000 steps. Day 71's attention LSTM, with the same training, ended at 1.000 at every length. So on this task, at this size and with plain SGD, the Transformer is not better: an honest result, and not the one the headline 'Transformers beat LSTMs' suggests. The LSTM reads the source in order, so its states carry a sense of 'how far from the end'; the Transformer has to work that out from positions alone.

WHY CHOOSE THIS?

Shows where the architecture's advantage stops.

WHY NOT THIS? (TRADEOFFS)

One size, one optimiser; Adam or warm-up (used in practice) were not tried.

4. Longer than any training sequence

DEFINITION — start here, no prior knowledge assumed

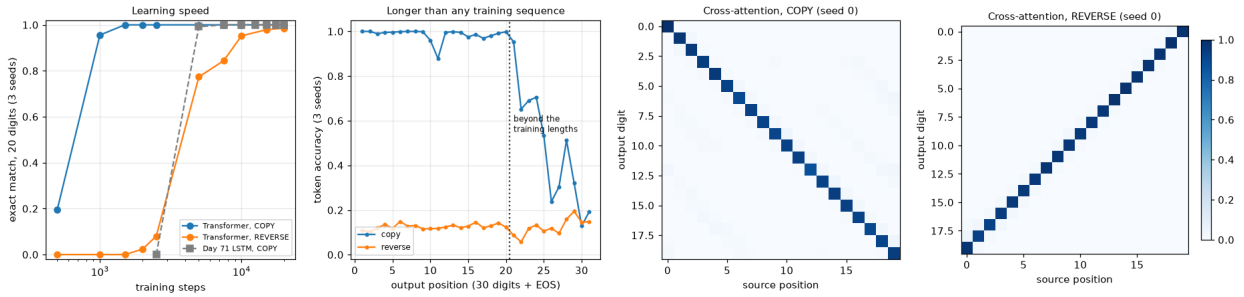
The trained models (3–20 digits) on 25, 30 and 40 digits. Token accuracy is split into positions 1–20, which training covered, and positions 21 on, which it never did.

REAL-WORLD SCENARIO

A clerk trained on forms of up to 20 lines is handed one of 30.

ACTUAL OUTPUT

```
task      length   exact  token acc   token acc, positions 1-20 / 21 on
copy          20   1.000      1.000
copy          25   0.000      0.897   0.986 / 0.719
copy          30   0.000      0.805   0.986 / 0.504
copy          40   0.000      0.638   0.916 / 0.388
reverse       20   0.986      0.994
reverse       25   0.000      0.134   0.124 / 0.171
reverse       30   0.000      0.125   0.126 / 0.121
reverse       40   0.000      0.161   0.135 / 0.184
Actual output from this lesson's run.
```



Left: exact match at 20 digits while training, with Day 71's LSTM on COPY. Second: token accuracy by position at 30 digits. Right: mean cross-attention weights on 20-digit tests (Section 5).

What it means

COPY keeps positions 1–20 almost perfect (0.986 / 0.719 at 25 digits) and gets worse past them: no exact matches, but token accuracy 0.897 at 25 digits against Day 71's 0.742. **REVERSE** falls to about chance everywhere (0.134 at 25 digits; one of 8 digits = 0.125), even at positions 1–20. A likely reason: the first output must read source position n − 1, and for n = 25 that position's encoding never appeared in training, so the model cannot find where the sequence ends; every later step is then lost too. This was not tested directly.

WHY CHOOSE THIS?

Separates what was learned as a rule from what was learned for the lengths seen.

WHY NOT THIS? (TRADEOFFS)

Sinusoidal positions alone do not give length generalisation; later models use relative positions.

5. What the cross-attention learned

DEFINITION — start here, no prior knowledge assumed

Day 72's measurements on the decoder's cross-attention: for each output digit, does the largest weight fall on the source position holding that digit, how much weight goes there, and how spread out the weights are (entropy; log 20 = 3.00 means spread evenly).

REAL-WORLD SCENARIO

Watching where a translator's eyes go on the page while they write each word.

MATHEMATICS

$$entropy = -\Sigma_i\, a_i \log a_i; \quad target = j \text{ (COPY) or } 19 - j \text{ (REVERSE)}$$

IMPLEMENTATION

```
A = cache[6][8][5][:, :20, :]
j = np.arange(20); target = j if task == "copy" else 19 - j
entropy = -(A * np.log(A + 1e-12)).sum(axis=2)
print(f"{task:<9}{(A.argmax(axis=2) == target).mean():>18.3f}{A[:, j,
    target].mean():>18.3f}{entropy.mean():>9.3f}")
```

ACTUAL OUTPUT

```
task       argmax on target  weight on target  entropy
copy                1.000             0.933    0.325
reverse             0.998             0.968    0.118
Actual output from this lesson's run.
```

What it means

Both models point at exactly the right source digit (argmax 1.000 and 0.998), with most of the weight there (0.933 and 0.968) and little spread (entropy 0.325 and 0.118). The heatmaps show the diagonal for COPY and the anti-diagonal for REVERSE. Inside the training lengths the model has learned the alignment the task needs, without being told it.

WHY CHOOSE THIS?

The weights can be read, and they match the task.

WHY NOT THIS? (TRADEOFFS)

One block, one head of cross-attention; deeper models are harder to read.

6. How sensitive is it to the learning rate?

DEFINITION — start here, no prior knowledge assumed

COPY, seed 0, stopped at 5,000 steps, at four learning rates.

REAL-WORLD SCENARIO

Trying four oven temperatures with the same recipe and the same timer.

ACTUAL OUTPUT

```
    lr    n=12    n=20
   0.1   1.000   1.000
   0.5   1.000   1.000
   1.0   1.000   1.000
   2.0   0.000   0.000
Actual output from this lesson's run.
```

What it means

Learning rates from 0.1 to 1.0 all reach 1.000 at 12 and 20 digits within 5,000 steps: COPY is forgiving. At lr 2.0 training fails completely (0.000 at both lengths). With clipping at norm 1.0, every step moves the weights by at most lr; a step of up to 2.0 is apparently too far for this model (what goes wrong inside was not examined). The safe range is wide, but it has an edge.

WHY CHOOSE THIS?

A quick check that lr 0.5 was not a lucky pick.

WHY NOT THIS? (TRADEOFFS)

One seed, one task; REVERSE was not swept.

7. What a training step costs

A batch of 32 sequences of 20 digits, forward and backward, against Day 71's model imported from its own script and timed in the same run, matrix library on one thread.

ACTUAL OUTPUT

```
Transformer: 25.7 ms per step; Day 71's attention LSTM: 24.2 ms; the LSTM takes 0.94 times as long
Actual output from this lesson's run.
```

In this run the Transformer took 25.7 ms per step and Day 71's LSTM 24.2 ms (ratio 0.94): about the same. Per step, the Transformer is not cheaper at this size; its advantage in Section 2 is that it needs far fewer steps. At 20 positions and d = 32 the attention matrices are small; their cost grows with the length squared, which would matter for long sequences. Only ratios within one run are meaningful.

Interview preparation — technical questions

Q: How is a Transformer decoder run at test time?

A: Greedily: start with GO, run the model, append the most probable token, repeat until the length (or EOS).

Q: How do you check that separately tested parts fit together?

A: A gradient check of the assembled model, entry by entry (1442 entries here).

Q: What does exact match measure that token accuracy does not?

A: Whether the whole sequence is right; one wrong digit makes it 0.

Q: How do you read cross-attention weights?

A: Take the decoder's cross-attention matrix: one row per output position, one column per source position.

Q: Why clip the gradient's norm?

A: One bad batch cannot make a huge step (Day 60); clipping at 1.0 kept Day 71's setup.

Interview preparation — theoretical questions

Q: Why can a Transformer learn COPY faster than an LSTM?

A: Attention reads any position in one step; an LSTM must carry information through the recurrence.

Q: Why is REVERSE harder for it?

A: The source position to read depends on the length, which positions alone do not give directly.

Q: Why do sinusoidal positions not give length generalisation?

A: Positions past the training lengths give encodings and score patterns the model never saw.

Q: Is a Transformer always better than an LSTM?

A: Not here: on REVERSE with plain SGD at this size, Day 71's LSTM ended perfect and the Transformer did not.

Key takeaway and what's next

> **KEY TAKEAWAY**
>
> Week 12's six exercise parts, copied without changes, make a working Transformer: the gradient checks, and trained exactly like Day 71's attention LSTM it learns COPY in a fraction of the steps, with cross-attention that finds the right digit. It is not better at everything: REVERSE is slower and ends just short of perfect, where the LSTM was perfect, and past the training lengths REVERSE collapses to chance.

- Gradient check: 1442 entries, largest difference 6.60e-11; 21642 weights against the LSTM's 17866.
- COPY: every length perfect from 1000-1500 steps (LSTM: 0.992 at 20 digits after 5000).
- REVERSE: 0.986 at 20 digits after 20000 steps; one seed in three perfect (LSTM: 1.000).
- Beyond 20 digits: no exact matches; COPY token accuracy 0.897 at 25, REVERSE about chance.
- Cross-attention: argmax on the right digit 1.000 (COPY) and 0.998 (REVERSE).

Suggested watch order, if starting from zero

- 1. Coding a Transformer from scratch, training and inference (Umar Jamil) — the whole model in code, trained and run.
- 2. Let's build GPT: from scratch, in code, spelled out (Andrej Karpathy) — a decoder-only Transformer built step by step, the direction Weeks 13–14 take.

Day 84 preview

- **Week 13 begins: tokenization.** Byte-pair encoding from scratch on real text: how a tokenizer learns subword pieces, why it never needs an unknown-word token, and what it still cannot handle.